

NEVS (National Employment Verification System) Project Status Report

Date: April 1, 2026

Current Phase: Phase 3 Completion / Entering Phase 4

Executive Summary

The NEVS Hyperledger Fabric project has successfully matured from a single-node prototype to a robust, multi-organization blockchain network utilizing advanced enterprise features. The core ledger, smart contracts, and REST API Gateway are now fully integrated, demonstrating secure cross-organizational consensus, isolated private data management, and dynamic service discovery.

Completed Milestones (Phases 1 - 3)

Phase 1: Foundation & Single-Org Setup

- **Network Initialization:** Bootstrapped the initial `nevs-network` utilizing a single organization (`CentralGovtMSP`) and a Raft orderer.
- **Base Chaincode:** Developed the initial Go smart contract to define basic Employment Records (Creation, Status Updates, History Tracking).
- **Dockerization:** Containerized peers, orderers, and CLI interfaces for localized testing.

Phase 2: Gateway Interoperability

- **REST API Middleware:** Developed `nevs-gateway` (Node.js/Express) integrating the `fabric-network` SDK to provide programmatic HTTP access to the ledger.
- **Connection Profiles:** Established dynamic connection configurations mapping the Gateway identity to the network nodes.
- **CRUD Operations:** Verified end-to-end functionality of querying and invoking basic ledger state modifications through the API.

Phase 3: Enterprise Architecture & Multi-Org Expansion (Recently Completed)

This was the most complex phase, elevating the network to production-ready enterprise standards:

1. Multi-Organization Integration (`CompanyOrgMSP`):

- Successfully generated and integrated cryptographic materials for a second organization (`CompanyOrg`).
- Symmetrically managed localized `msp` and `tls` directories (resolving mTLS "Cold War" disconnection anomalies).
- Executed surgical `configtxlator` channel configurations to natively bind `CompanyOrgMSP` into the `nevs-channel` without corrupting existing read/write policies.

2. Chaincode as a Service (CCaaS):

- Decoupled smart contract execution from the core peers by transitioning to the CCaaS model.
- Deployed shared `employment` chaincode containers capable of servicing requests from both `CentralGovt` and `CompanyOrg` peers simultaneously.

3. Dual-Endorsement Consensus:

- Enforced strict decentralized trust by applying a dual-endorsement policy:
`AND( 'CentralGovtMSP.member', 'CompanyOrgMSP.member' )` .
- Verified that write operations (e.g., standardizing an employment record) are cryptographically signed and independently verified by both organizations before ledger commitment.

4. Private Data Collections (PDC):

- Implemented an `employmentPrivateData` collection to secure highly sensitive attributes (Salary, Compensation).
- Mapped the SDK Gateway to transmit PDC fields as payload **Transient Data**, ensuring sensitive fields route directly to Endorsers over TLS, bypassing block inclusion and Orderer visibility entirely.
- Verified public/private read isolation across standard and private API schemas.

5. Dynamic Service Discovery:

- Configured the SDK Gateway to automatically detect endorsing peer layouts across the multi-org fabric matrix.
- Verified the Gateway can dynamically route proposals to `peer0.company.org` alongside government peers based on the parsed network channel configuration.

6. Automated Recovery:

- Authored the `restore_full_network.sh` automation script. This heavily streamlines operations by reliably spinning up the entire dual-org network, channel binding, anchor peer mapping, CCaaS packaging, and dual-endorsement approval sequencing in a single execution.

Current Architecture State

Component	Status	Details
Orderer	Active	Raft Consensus (orderer.nevs.gov:7050)
Central Govt Peer	Active	peer0.central.govt:7051 & peer1.central.govt
Company Peer	Active	peer0.company.org:9051 (Joined Phase 3)
Gateway Service	Active	Port 3000 (Dynamic Discovery Enabled)
Smart Contract	Active	employment (CCaaS Mode, Sequence 3)

Gateway Interface State (nevs-gateway)

The Node.js/Express gateway acts as the centralized middleware bridging external clients to the Fabric network. It is currently operating flawlessly with the following enterprise capabilities:

Core Capabilities

- **Fabric SDK Integration:** Utilizes `fabric-network` to securely connect to the channel using X.509 cryptographic identities.
- **Dynamic Endorsement Discovery:** Automatically parses network topology via Fabric Service Discovery, seamlessly routing transactions to `peer0.company.org` and `peer0.central.govt` to mutually satisfy the dual-endorsement policy.
- **Transient Data Routing:** Extracts sensitive fields (Salary, Compensation) from standard JSON payloads and injects them as transient data bytes prior to proposal generation, maintaining complete Orderer abstraction for PDC rules.

Active REST API Endpoints

Endpoint	Method	Chaincode Function mapped	Details
/api/employment/	POST	CreateEmploymentRecord	Commits basic data & securely passes Transient Salary Data.
/api/employment/	GET	GetAllEmploymentRecords	Retrieves public indices from the main

Endpoint	Method	Chaincode Function mapped	Details
			ledger state.
/api/employment/employee/:id	GET	GetEmploymentHistory	Yields the chronological provenance timeline for a worker.
/api/employment/record/:emp/:rec	GET	GetEmploymentRecord	Fetches a specific public record configuration (omits PDC).
/api/employment/:emp/:rec/status	PUT	UpdateEmploymentStatus	Pushes validated state changes to the active ledger.
/api/employment/private/:rec	GET	GetPrivateEmploymentData	Evaluates the explicitly authorized employmentPrivateData collection.

Identified Priorities for Next Steps (Phase 4)

With the backend Fabric infrastructure mathematically proven and operationally stable, subsequent efforts should pivot toward integration and operational stability.

1. Frontend Integration (HireNest UI):

- Wire the React/Frontend applications directly into the `nevs-gateway` APIs.
- Expose the PDC (Salary/Compensation) dynamically via UI permission layers.

2. CouchDB & Rich Queries (Optional but Recommended):

- To support complex UI dashboards, transition the Peer state DB from LevelDB to CouchDB to enable advanced JSON-based querying.

3. Event Listeners / Subscriptions:

- Implement Fabric Event Listeners in the Gateway to stream real-time block/transaction notifications to the UI via WebSockets or SSE.

4. Hardware Deployment:

- Prepare the `docker-compose` files and volume bindings for final deployment onto the targeted hardware (Raspberry Pi/LAN environment).

Report generated via Antigravity Agent Diagnostics.